Vroomie
How Vroomie Listens

What the product does, why it can be trusted, and where its limits are

For readers with no engineering background
github.com/DGeorgeA/vroomie-app · branch main
Live at vroomie.in

All performance figures herein are measured results on held-out data, reproducible with the committed test harnesses.

1. The thirty-second version

A mechanic with thirty years' experience can stand next to a running engine, listen, and say *"that's a bearing going."* They aren't measuring anything. They've simply heard that sound enough times to recognise it.

Vroomie does the same thing, with a phone.

The driver points their phone at the engine. Within a few seconds the app says whether it sounds healthy — and if not, what it most likely is, in plain words, with a report they can hand to a workshop.

No sensors. No plug-in device. No garage visit. **The phone they already own.**

2. The mistake almost everyone makes

It's easy to build something that listens and guesses. It's very hard to build something you can *trust*.

Trust has to work in two directions:

- Tell a driver their perfectly fine car is broken → they stop believing the app
- Miss a real fault → the app is worthless

Most naive attempts fail the first test badly, and Vroomie's earliest version was no exception. It would hear a **television** playing in the background and confidently report a power-steering fault.

Understanding *why* is the key to understanding the whole product.

The trap

Imagine you have photographs of ten faulty engine parts, and someone hands you a photo of a cat. If your only question is *"which of my ten pictures does this most resemble?"* — you will answer with one of them. Perhaps the frayed belt. You have no way to say *"none of these."*

That was the flaw. The software asked **"which fault does this sound most like?"** That question always returns a fault, because everything resembles something. Silence has a nearest match too.

The fix

Vroomie asks a different question:

> **"Is this closer to a known fault than it is to a healthy engine?"**

The change is small in words and total in effect. Vroomie stores recordings of **healthy** engines alongside the faulty ones. A fault is only reported when the sound is measurably closer to a fault than to any healthy engine — or to speech, music, traffic, or a fan.

That is why the app can say *"nothing wrong"*. And it's why a television no longer sets it off: a television sounds far more like a television than like a failing bearing.

This is the single most important thing to understand about the product. Everything else is engineering built on top of it.

3. Two ways of listening

Vroomie runs two systems at once. They have deliberately opposite strengths.

The fast identifier — "I know this exact recording"

This works the way **Shazam** does.

Think of a piece of music as a night sky. Most of it is dark, but there are bright stars. Shazam ignores everything except the brightest points and records their pattern — *this star here, that one a moment later, up there.* A song heard in a noisy café still has the same star pattern, so it still matches.

Vroomie does exactly this with engine sounds. It reduces the noise to its most distinctive peaks and checks whether they line up **in time** with a recording it already knows.

That timing requirement is what makes it trustworthy. Random noise might accidentally produce a few matching points, but it cannot produce hundreds of them *in the correct order and spacing*. A genuine match aligns like a key in a lock.

This is what identifies a known sample in about **three seconds**.

Its honest limitation: it recognises *that specific recording*. Shazam identifies the studio track but can't recognise your friend humming the tune. Likewise the fast identifier recognises Vroomie's reference recording, not a different car with a similar fault.

The pattern matcher — "this sounds like the family of sounds a failing bearing makes"

This is the general-purpose engine, and it's the one that works on real customer vehicles.

It uses **YAMNet**, a sound-recognition model trained by Google on millions of clips of the everyday world — speech, engines, rain, machinery, music. Vroomie uses it as a translator: it converts any sound into a compact numerical description of *what kind of sound it is*, then compares that description against its library of faults **and** its library of healthy engines.

It generalises to cars it has never encountered. It is, correspondingly, less certain than the fast identifier — which is why it reports a confidence level rather than a verdict.

Why both

	Fast identifier	Pattern matcher
Answers	"Is this exact recording playing?"	"Does this sound *like* a known fault?"
Speed	~3 seconds	The full recording
Works on	Known reference recordings	Any vehicle with a supported fault
Certainty	Very high	Graded, 65–97%

One gives certainty on what it knows. The other gives reach into the real world. Neither alone is sufficient.

4. Four gates before anyone is told anything

A sound must pass all four checks. Any one of them can stop it.

1 · Is it loud enough? Silence is discarded immediately.

2 · Is this a vehicle at all? Speech, television, music and street noise are filtered out here. This gate is why the app is quiet in an ordinary room.

3 · Is it more fault-like than healthy-like? The comparison from Section 2 — the heart of the system.

4 · Did it last? A fault has to dominate the recording, not appear for one stray moment. A single suspicious instant never produces a diagnosis.

Think of it as four colleagues who must all agree before anyone speaks to the customer.

5. Confidence you can act on

Vroomie never announces *"your car is broken."* It states how sure it is, and the levels mean specific things:

It says	It means
97%	Recognised a known recording almost exactly
70–97%	Confirmed fault pattern, sustained through the recording
65–69%	*Possible* — flagged explicitly as needing a workshop to verify
Nothing	No reliable signal. The app stays silent rather than guess.

That last row is a feature, not a gap. **A diagnostic tool that always finds something is worthless.** The willingness to say nothing is what makes the other rows believable.

The 65–69% band was only added after measuring that it produced **zero extra false alarms** across 140 recordings of healthy vehicles.

6. How we know it works

Testing is done against recordings the system has **never seen while being built** — the equivalent of examining a student on questions not in the textbook.

The main stress test plays every reference sound through every realistic condition: four different simulated speaker-and-room environments, four volume levels from loud to very quiet, and three different points to start listening. That's **384 combinations**.

Test	Result
Correct identification across all 384 conditions	**384 / 384 — 100%**
Wrong fault named	**0**
Healthy engines falsely alarmed (fast identifier)	**0 out of 140**
Speech, music, fan, traffic, silence falsely flagged	**0 out of 9**
Time to identify	**about 3.7 seconds**

The weaknesses, stated plainly

We publish these because an investor will find them anyway, and a founder who names them first is more credible than one who doesn't:

- **Roughly 1 in 23 healthy cars** may see a low-confidence alert from the pattern matcher.
- **About two in three** genuinely unseen faults are detected — not all of them.
- **Nine fault types** are supported today. Some common faults have no reference recording yet and simply cannot be detected.

Here is what matters about all three: **they are limited by how many recordings we have, not by how the software works.** Every one improves as real recordings from real vehicles are added — with no changes to the code. The machinery to absorb new recordings is already built and automatic.

That is precisely why the investment is in data and computing capacity rather than in more engineering.

7. The one thing that must be right on demo day

The app listens through a microphone. Anything that interferes with that microphone will defeat it, no matter how good the analysis is.

The lesson came from a real failure. During an investor call over Zoom, the samples were played through the laptop speaker while the same laptop's microphone was listening. Nothing was detected.

The cause was Zoom, doing its job correctly. Every video-calling app removes your own speaker output from your microphone — otherwise the person on the other end hears themselves echo. Zoom recognised the played sample as the laptop's own sound and **deliberately subtracted it** before the app could hear it. Vroomie was handed audio with the exact thing it was listening for removed.

The app's records prove it: the same sound scored **79** during the call and **736** outside it, on identical software.

The rule that follows: run Vroomie on a **phone**, keep the video call on the laptop. The phone's microphone belongs to Vroomie alone. Never let a calling app own the microphone Vroomie is using.

8. Why this is hard to copy

The data compounds. Every fault recording collected makes detection better for every future user. A competitor starting today starts from nothing, and the gap widens with every diagnosis.

The discipline is the moat. This architecture is the product of repeatedly finding subtle failures and fixing them with measurement rather than intuition — a library too heavily weighted toward one fault, distortion introduced by cheap speakers, thresholds tuned on simulations instead of real phones. Each fix is documented alongside the measurement that justified it. That accumulated judgement is far harder to replicate than the algorithm.

It costs nothing to run. Analysis happens on the handset. A million diagnoses cost the same to operate as a thousand.

Failures are visible. Every session — including the ones that fail — records why. That is how the Zoom problem was diagnosed precisely, two days later, without touching the machine. Most products in this space cannot tell you why they failed.

Every figure in this document comes from automated tests that anyone can re-run against the codebase. Technical detail: `VROOMIE_PIPELINE_TECHNICAL.md`*.*